

Day 8 - Responsive Design: Media Queries, Units, Mobile-First

Day 8 — Responsive Design: Media Queries, Units, Mobile-First

Objectives

- Understand what "responsive" actually means and why it's non-negotiable for any real website today
- Use media queries to apply different CSS at different screen widths
- Understand the difference between mobile-first and desktop-first, and why mobile-first is the modern default
- Choose the right unit for the job: px, %, em, rem, vw, vh
- Understand the viewport meta tag and why every page needs it

What does "responsive" actually mean?

A **responsive** web page is one that adapts its layout to look good and remain usable across a wide range of screen sizes — a huge desktop monitor, a laptop, a tablet, and a phone, all viewing the exact same page, all getting a layout appropriate to their screen. This isn't optional polish anymore — a large share of all web traffic, for most sites, comes from phones, and a page that only works well on a wide desktop screen is, in a very real sense, broken for a huge portion of potential visitors.

The viewport meta tag — required on every page, without exception

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

Without this one line, mobile browsers assume your page was designed for a wide desktop screen and automatically zoom out to fit that assumed-wide layout onto the phone's small screen, rendering everything tiny and requiring the visitor to manually pinch-zoom just to read anything. This meta tag tells the browser: "the width of this page should match the actual width of the device's screen, and start at normal (1.0) zoom" — which is what allows your responsive CSS (media queries, below) to actually take effect the way you intend. **Add this to the `<head>` of every single page you build, starting today, for the rest of this course and beyond.**

Media queries — applying CSS conditionally, based on screen width

```
.container {
```

```

    display: flex;
    flex-direction: row;
}

@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}

```

A **media query** wraps a block of CSS rules in a condition — here, "only apply this if the browser's viewport width is 768 pixels or less." Outside that condition (on a wider screen), the original `flex-direction: row` rule from earlier in the file still applies; once the screen narrows to 768px or below, the media query's rule kicks in and overrides it, stacking the flex items vertically instead — a very common real pattern for adapting a side-by-side layout into a stacked one on a phone.

`max-width` matches "this width or narrower"; the less common `min-width` matches "this width or wider" — which one you reach for depends on which overall strategy you're using, explained next.

Mobile-first vs. desktop-first — and why mobile-first is the modern default

Desktop-first means you write your default CSS assuming a large desktop screen, then use `max-width` media queries to override and simplify the layout for progressively smaller screens. **Mobile-first** means the reverse: your default (un-wrapped) CSS assumes a small phone screen, and you use `min-width` media queries to progressively **add** complexity as the screen grows larger:

```

/* Mobile-first: this is the DEFAULT, unwrapped CSS -- assume a phone screen */
.container {
  display: flex;
  flex-direction: column;
}

/* THEN, only once the screen is at least 768px wide, switch to a row layout */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}

```

Mobile-first is the modern, generally recommended default, for a couple of concrete, practical reasons: it forces you to design the simplest, most essential version of your page first (What does this page actually need, at its core, once you strip away extra decoration that only fits on a spacious screen?), and it tends to produce less CSS overall, since you're only **adding** rules for extra space as it becomes available, rather than writing a complex desktop layout and then fighting to **undo** parts of it for a phone.

A common set of **breakpoints** (the specific widths at which your layout changes) you'll see frequently in real projects, though these are conventions, not hard rules enforced by CSS itself: roughly 576px (small phones), 768px (tablets), 992px (small laptops), 1200px (large desktops) — you'll choose your own based on where your **specific** design actually starts looking cramped or awkwardly spaced out, rather than blindly copying these numbers.

Choosing the right unit for the job

- **px** (pixels) — an absolute, fixed unit. Predictable, but doesn't scale with anything else — good for things that genuinely should stay a fixed size regardless of context, like a **1px** border.
- **%** (percentage) — relative to the **parent** element's size. **width: 50%** means "half of whatever my parent's width happens to be" — genuinely useful for fluid layouts, but its meaning changes depending on what the parent is, which can get confusing in deeply nested layouts.
- **em** — relative to the **current element's** font size (or, if the element itself doesn't set one, its nearest ancestor's). **2em** means "twice the current font size." Because **em** compounds through nested elements (an element's **em** is based on its own computed font size, which might itself have been set in **ems** relative to something else), it can produce surprising, hard-to-predict results in deeply nested structures.
- **rem** ("root em") — relative to the **root** element's font size (**<html>**, which defaults to 16px in essentially every browser unless you change it) — always, no matter how deeply nested the element using it is. This predictability is exactly why **rem** is strongly preferred over **em** for most sizing in modern CSS: **1.5rem** always means "1.5 times the root font size," full stop, regardless of nesting.
- **vw/vh** ("viewport width/height") — relative to the **browser window's** current width/height. **100vw** means "the full width of the browser window," and **50vh** means "half the height of the browser window" — genuinely useful for things that should scale directly with the screen itself, like a full-screen hero section (**height: 100vh;**).

A very common, sensible modern convention: use **rem** for font sizes and most spacing (so that if a visitor increases their browser's default font size for accessibility reasons, your whole layout scales proportionally with it), **%** or **fr**/Grid-Flexbox tools for fluid container widths, and **px** only for things that genuinely should never scale, like a thin border.

Testing responsiveness with DevTools

Every browser's DevTools includes a **device toolbar** (usually a small phone/tablet icon in the DevTools toolbar, or Ctrl+Shift+M) that lets you preview your page at different screen widths — including specific real device presets (iPhone, iPad, and others) — without needing an actual physical phone to test on. Get in the habit of opening this and dragging the width slider (or picking device presets) every time you build something responsive, checking that your media query breakpoints actually kick in where you expect and that nothing looks broken or overlapping at any width in between.

Exercises

Open **starter.html** and **starter.css**, follow the **<!-- TODO --> /* TODO */** markers, and check your work using DevTools' device toolbar, comparing against **CHECKLIST.md**.